

C++26: the new debugging header

Sandor Dargo on Sep 30

3 min read

If you’ve ever needed to hit a breakpoint programmatically in C++, you’ve probably written something like this:

CPP

```
1  #if defined(_MSC_VER)
2      __debugbreak();
3  #elif defined(__clang__)
4      __builtin_debugtrap();
5  #elif defined(__GNUC__) && (defined(__i386__) || defined(__x86_64__))
6      __asm__ volatile("int3");
7  #else
8      raise(SIGTRAP);
9  #endif
```

Every major C++ project reinvents this wheel. Boost.Test has `debugger_break()` , Catch2 has `CATCH_BREAK_INTO_DEBUGGER` , Dear ImGui has `IM_DEBUG_BREAK()` and Unreal has `UE_DEBUG_BREAK` . The platform knowledge required is arcane but well-known to tooling implementers — it just wasn’t standardized.

C++26 fixes this with [P2546R5](#) and its companion [P2810R4](#) thanks to Rene Ferdinand Rivera Morell. A new `<debugging>` header provides three functions.

The three functions

CPP

```
1  #include <debugging>
2
3  namespace std {
4      void breakpoint() noexcept;
5      bool is_debugger_present() noexcept;
6      void breakpoint_if_debugging() noexcept;
7  }
```

All three are `noexcept` and all three are freestanding. None are `constexpr` — debugging is a runtime concern.

`std::breakpoint()`

An unconditional breakpoint. When a debugger is attached, execution halts and control is handed to the debugger. On x86, this typically compiles to a single `int3` instruction.

When no debugger is present, the behaviour is implementation-defined — on most platforms the process receives a signal (such as `SIGTRAP`) and will terminate unless the signal is caught.

`std::is_debugger_present()`

Returns `true` if the program is currently being traced by a debugger. The check is performed immediately on each call — the result is not cached because a debugger can attach or detach at any time during execution.

Linux, GCC's `__has_include` reads `__has_include` status and checks the `__has_include` field. The standard doesn't mandate any particular mechanism — the semantics are implementation-defined, with the intended behaviour described in non-normative notes.

What makes this function interesting is that it's **replaceable**, in the same sense as `operator new` and `operator delete`. You can define your own version:

CPP

```
1 bool std::is_debugger_present() noexcept {
2     return my_hardware_debug_flag;
3 }
```

This was introduced by [P2810R4](#) to solve the freestanding problem: on bare-metal platforms, there may be no OS-level way to detect a debugger. Without replaceability, the function would either have to be excluded from freestanding (forcing `#ifdef` guards everywhere) or return a useless default. Replaceability lets you wire it to whatever your platform provides.

```
std::breakpoint_if_debugging()
```

The convenience function most users will reach for. It's equivalent to:

CPP

```
1 if (std::is_debugger_present())
2     std::breakpoint();
```

When a debugger is attached, you hit a breakpoint. When it's not, nothing happens. This is the pattern that Catch2, Boost.Test, and every other framework implements manually.

Beyond breakpoints

Having `is_debugger_present()` as a separate function enables more than just conditional breakpoints. You might use it to:

- Produce extra diagnostic output when debugging
- Disable timeouts that would fire during a stepping session
- Switch to a more debugger-friendly allocator
- Skip optimizations that make debugging harder

The separation of concerns is deliberate — `breakpoint()` and `is_debugger_present()` are orthogonal building blocks, and `breakpoint_if_debugging()` is the convenience composition.

Conclusion

`<debugging>` is another one of those C++26 additions that makes you wonder why it took this long. The platform knowledge is well-understood, every major project already has its own version, and the API is three functions. Sometimes the most useful standard library additions are the simplest ones.

Connect deeper

If you liked this article, please

- hit on the like button,
- [subscribe to my newsletter](#)
- and let's connect on [Twitter](#)!
- if you're preparing for a C++ quant/trading interview, check out [GetCracked](#)